

Movies Data Manager

Rayhab Sajjad
FAST NUCES, Islamabad
BS Data Science
Islamabad
i242586@isb.nu.edu.pl

Alisha Razzaq
FAST NUCES, Islamabad
BS Data Science
Islamabad
i242616@isb.nu.edu.pk

Abstract—This project implements a Movies Data Manager using core data structures without STL. The system processes the IMDb 5000 Movie Dataset and organizes movie information using manually implemented structures such as AVL Trees, Linked Lists, Hash Tables, and Graphs. The project focuses on efficient searching, organization, and recommendation of movies by leveraging structural relationships between movie attributes.

Index Terms—Data Structures, AVL Tree, Graph Traversal, Hash Table, Linked List, Movie Recommendation System

I. INTRODUCTION

With the rapid growth of digital content, managing and searching large datasets efficiently has become critical. Movie databases like IMDb contain thousands of records with multiple attributes, making structured storage and fast access necessary.

This project designs a Movies Data Manager using fundamental data structures implemented manually in C++. It reads the IMDb 5000 Movie Dataset and supports searching, recommendations, and relationship discovery between movies based on shared attributes such as actors and genres.

II. MOVIES DATA MANAGER: DESIGN AND IMPLEMENTATION

The development followed a structured pipeline: data extraction, cleaning, organization, and relationship modeling. Core data structures were implemented manually in C++ without STL.

A. A. Data Extraction and Cleaning

Relevant attributes such as title, year, genres, actors, director, and IMDb score were extracted from the dataset. Each row was parsed into a `MovieNode`, with cleaning applied to remove non-printable characters, handle missing values, and ensure case-insensitive comparisons. Duplicate titles were prevented during insertion.

B. B. Central Storage Using AVL Tree

Movies are stored in an AVL Tree using titles as keys. The self-balancing property ensures logarithmic time complexity for insertion and search. The tree supports rotations (LL, RR, LR, RL), insertion, deletion, and in-order traversal, all encapsulated in the manager class.

C. C. Hash Table Implementation with Chaining

An `ActorHashTable` with 2048 buckets uses separate chaining and a polynomial rolling hash function (base 31, modulo 2147483647) for case-insensitive actor indexing. It supports:

- **find(name)**: Locate exact matches.
- **findPartial(query)**: Find the first actor containing the query.
- **getOrCreate(name)**: Retrieve an existing actor or create a new one.

D. D. Linked List Structures

Bidirectional relationships are maintained via linked lists:

- **Genre List**: Each movie has a linked list of genres.
- **Actor List in MovieNode**: Points to associated actors.
- **Movie List in ActorNode**: Represents an actor's filmography.

E. E. Graph Construction for Relationships

Each movie is a vertex; edges connect movies sharing actors or genres. The graph uses a hash table with linear probing. Edges are labeled with the shared attribute (actor or genre). Actor-based edges connect movies within an actor's filmography, while genre-based edges connect movies sharing at least one genre.

F. F. Algorithms Used

Core algorithms implemented include:

- AVL Tree insertion and balancing
- Polynomial rolling hash for actors
- Separate chaining for hash table collisions
- Linear probing for graph vertices
- BFS for recommendations and shortest path

G. G. Breadth-First Search Algorithm

Algorithm 1: Breadth-First Search for Movie Recommendations

Input: Graph $G(V, E)$, starting movie vertex s , maximum depth d
Output: Set of recommended movies R
Create empty queue Q and boolean array $Visited$
Initialize recommendation array R
 $distance[s] \leftarrow 0$
Enqueue s and mark visited
while Q not empty **and**
 $|R| < maxRecommendations$ **do**
 $v \leftarrow Dequeue$
 $currentDist \leftarrow distance[v]$
 if $0 < currentDist \leq d$ **then**
 Add v to R with calculated score
 end
 if $currentDist < d$ **then**
 for each neighbor u of v **do**
 if u not visited **then**
 Enqueue u ; mark visited;
 $distance[u] \leftarrow currentDist + 1$
 end
 end
 end
 end
end
Sort R by score; return top k

H. H. Hash Table Lookup Algorithm

Algorithm 2: Hash Table Lookup with Separate Chaining

Input: Hash Table H , actor name $name$, capacity cap
Output: ActorNode pointer or null
Compute polynomial rolling hash of $name$
 $index \leftarrow hash \bmod cap$
 $current \leftarrow H.buckets[index]$
while $current \neq null$ **do**
 if $caseInsensitiveEqual(current.actor.name, name)$ **then**
 return $current.actor$
 end
 $current \leftarrow current.next$
end
return $null$

III. RESULTS

The project implemented various operations on a graph-based data structure to manage relational information between movies and actors.

For instance, the shortest path between two nodes was determined using Breadth-First Search (BFS). BFS was chosen because it explores the graph level by level, ensuring the path with the fewest connections is found. Other algorithms, such as Dijkstra or Bellman-Ford, were not used since the graph is

unweighted, making BFS a simpler and more efficient solution for this context.

Additional operations, including insertion, deletion, and search, successfully updated and traversed the graph, demonstrating the correct implementation of core data structure functionalities.

Overall, the results indicate that the system efficiently manages nodes and their relationships while correctly performing queries and updates.

IV. DISCUSSION

The project provided an opportunity to apply graph data structures to manage relational data efficiently.

Through implementing various operations, the students gained insight into the importance of selecting the appropriate algorithm for the problem context, such as using BFS for shortest path queries in unweighted graphs.

The project also highlighted practical skills, including handling dynamic insertion and deletion, maintaining connections between nodes, and ensuring correct traversal of the graph.

Overall, the project reinforced the understanding of how data structures and algorithms are applied in real-world scenarios, demonstrating both theoretical and practical learning outcomes.

V. CONCLUSION

The Movies Data Manager successfully implements AVL trees, hash tables, linked lists, and graph structures manually. It supports efficient searching, recommendations, and shortest path queries on the IMDb 5000 Movie Dataset.

A. A. Summary

The system processes all movies, maintains relationships, and supports queries without STL usage.

B. B. Key Features

- AVL Tree for balanced movie storage
- Hash table with separate chaining for actors
- Linked lists for relationships
- Graph representation for movie connections
- BFS-based recommendation and shortest path algorithms